

Backend de Sably en Cloudflare

D1, R2, Pages Functions y el panel de administración

Connexis

8 de agosto de 2026

Índice

Backend de datos dinámicos sobre Cloudflare	1
El panel: sably.co/admin	1
Por qué Cloudflare y no Supabase	2
Lo que hay montado	2
La API	3
Cumplimiento legal	3
Lo que falta	4
Operar el día a día	5
Migrar a Laravel después	5

Backend de datos dinámicos sobre Cloudflare

Estado a 2026-08-07. **Desplegado y probado en producción.** Base D1 `sably-pulso`, API en `/api/v1`, panel en `/admin`. Falta crear la aplicación de Cloudflare Access y cablear el frontend.

El panel: `sably.co/admin`

Hay panel con cinco secciones —resumen, moderación, leads, contenido y despliegue— y se entra con **Cloudflare Access**, no con una contraseña propia.

La diferencia importa: Access ya resuelve identidad, caducidad de sesión y revocación, es gratis hasta 50 usuarios y admite Google, GitHub o un código de un solo uso por correo. Montar usuarios, contraseñas, recuperación y segundo factor sería superficie que mantener y que se puede comprometer, para un sistema que usa una persona.

Al origen llega un JWT firmado en la cabecera `Cf-Access-Jwt-Assertion` con el correo de quien entró, y eso es lo que se guarda en `moderation_log`: **la auditoría sale sin escribir una línea de código de sesiones.**

La firma se verifica contra el JWKS del equipo. No es opcional: la cabecera es texto que cualquiera puede enviar si llega al origen saltándose Access, así que sin verificarla el panel estaría abierto a quien conozca la URL.

Sin subdominio

Access protege una ruta igual de bien que un host, y `/admin` evita otro proyecto de Pages, otro certificado y otra entrada de DNS.

□ Un detalle que el diseño marcó como trampa: **el enlace del correo abre una página de confirmación, no ejecuta el cambio.** Los clientes de correo hacen `prefetch` de los enlaces, así que un

GET que aprobara directamente aprobaría comentarios solo con que Gmail precargara el mensaje. Se aprueba con un POST desde esa página.

Cuando llegue Filament (Fase 2, Laravel 13) ahí sí habrá panel con login, y será el sitio natural para moderar. Este backend es puente: el contrato ya nace en `/api/v1` con los nombres y verbos que tendrá Laravel, y las tablas usan clave primaria ULID para mapear 1:1 a Eloquent con HasUlds.

Por qué Cloudflare y no Supabase

Tú ya lo habías decidido, y los números lo respaldan para este caso concreto:

	Cloudflare	Supabase
Origen	El mismo que sably.co ☐ sin CORS ni preflight	Dominio aparte ☐ CORS y una petición extra
Infra	Pages, R2 y el token ya están en el proyecto	Un proveedor y una factura más
Latencia	Ejecuta en el borde, junto al visitante	Región fija, salto transatlántico desde LATAM
Coste a este volumen	0 USD en free tier	0 USD hasta 500 MB, luego 25 USD

La ventaja real no es el precio, es que **las Pages Functions comparten origen con el sitio**. Eso elimina CORS, elimina la comprobación de `Origin` como control de seguridad y elimina un dominio que mantener.

Lo que hay montado

Base de datos: `sably-pulso`

0a173a39-b9cb-4d50-a865-b8465af5bad6 · 10 tablas, 4 vistas, 13 índices.

Tabla	Para qué
<code>subject</code>	Lista blanca de lo votable. Sin fila aquí, no se puede votar ni comentar
<code>consent</code>	La autorización con el texto literal que se mostró
<code>vote</code>	Votos de artículo, uno por visitante
<code>comment</code>	Comentarios, nacen en pending
<code>lead</code>	Formulario de contacto
<code>purchase</code>	Compras verificadas de Hotmart
<code>review_token</code>	Token de un solo uso para dejar reseña
<code>course_review</code>	Reseñas de curso, atadas a una compra
<code>moderation_log</code>	Quién aprobó qué y cuándo

Las dos decisiones que sostienen todo

1. No se almacenan promedios. Ni una columna con el rating medio, ni triggers que la mantengan. El promedio sale de una vista SQL sobre la tabla de votos, cada vez.

Un contador almacenado hay que actualizarlo en cada **INSERT**, **UPDATE** y **DELETE**, y basta olvidar un caso —borrar una reseña ya aprobada, por ejemplo— para publicar un número que no corresponde a ninguna fila. Sin contador no hay nada que desincronizar. A esta escala calcularlo es gratis.

2. Lo que lee Google se hornea en el build. El JSON-LD y las estrellas visibles viajan en el HTML servido, nunca se inyectan por JavaScript. Así el marcado es auditable abriendo `dist/`, y no depende de que un fetch responda a tiempo cuando pasa Googlebot.

La garantía contra lo que ya pasó

Este sitio publicó valoraciones inventadas: un hash del slug generaba “4.7 de 5 · 87 valoraciones” y eso alimentaba datos estructurados. El esquema está diseñado para que no pueda repetirse:

```
purchase_id TEXT NOT NULL UNIQUE REFERENCES purchase(id)
```

Una reseña sin compra detrás **no es un caso que validar en código: es una fila que la base rechaza**. Y Article no recupera `aggregateRating` nunca — no es cuestión de umbral, es que Google no da fragmentos de reseña para ese tipo, que es exactamente lo que reportó Search Console.

Para que Course vuelva a mostrar estrellas hacen falta **5 reseñas aprobadas a mano, cada una atada por clave foránea a una compra real de Hotmart**.

La API

Todo bajo `https://sably.co/api/v1/`, mismo origen que el sitio.

Método	Ruta	Qué hace	Protección
POST	<code>/votos</code>	Vota 1–5 un artículo	Turnstile, límite por IP, 3 s mínimo de lectura
GET	<code>/pulso?subject=</code>	Devuelve votos y promedio	Pública
POST	<code>/comentarios</code>	Deja comentario (queda pendiente)	Turnstile, límite por IP, puntuación de spam
GET	<code>/comentarios?subject=</code>	Comentarios aprobados, sin correos	Pública
POST	<code>/leads</code>	Formulario de contacto	Turnstile, dedupe por correo y día
GET	<code>/snapshot</code>	Todo lo publicable, para el build	Bearer SNAPSHOT_TOKEN
POST	<code>/subjects</code>	Siembra el catálogo	Bearer SNAPSHOT_TOKEN

Cómo se protege sin cuentas de usuario

- **Turnstile** de Cloudflare, que no pide resolver puzzles. El script se carga solo al abrir el formulario: en el layout serían ~70 KB contra un presupuesto total de 50 KB de JS.
- **Límite por IP** contando sobre la propia tabla destino: 5 comentarios/hora, 10 leads/hora, 30 votos/hora.
- **Tiempo mínimo de lectura** como CHECK de la base, no como if del endpoint: un voto a los 400 ms no se puede insertar ni aunque el código falle.
- **La IP no se almacena**. Se guarda `SHA-256(ip + sal + día)`, que permite exigir IPs distintas y limitar por origen sin conservar un dato personal, y que al cambiar cada día tampoco sirve para seguir a nadie.

Cumplimiento legal

La Ley 1581/2012 colombiana obliga a **poder probar** la autorización, no solo a pedirla, y el RGPD art. 7.1 dice lo mismo. Por eso consent guarda la versión de la política y **el texto literal que vio la persona**, no un booleano.

purge_after fija el plazo de conservación por fila: 90 días para el correo de quien comenta, 2 años para un lead comercial. Un cron los borra, así que el cumplimiento no depende de que alguien se acuerde.

Lo que falta

Paso	Estado
Base D1 creada y migrada	☐ hecho
7 endpoints de API	☐ desplegados y probados en producción
Panel de 5 secciones	☐ desplegado, rechaza sin autenticar
Binding D1 en el proyecto de Pages	☐ producción y preview
IP_SALT y CF_ACCESS_TEAM_DOMAIN	☐ configuradas
Deploy hook para el botón de reconstruir	☐ creado
Aplicación de Cloudflare Access	☐ te toca : el token no tiene ese permiso
CF_ACCESS_AUD (sale al crear la app)	☐ te toca
TURNSTILE_SECRET y clave de sitio	☐ te toca
DEPLOY_HOOK_URL con la URL del hook	☐ te toca
Cablear ArticleRating y LeadModal	☐ pendiente
Formulario de comentarios	☐ pendiente
Snapshot y horneado en el build	☐ pendiente
Webhook de Hotmart y reseñas	☐ pendiente de verificar el webhook

Lo que se verificó en producción

POST /api/v1/votos	→ {"ok":true,"votes":1,"avg":5}	el voto entra en D1
GET /api/v1/pulso	→ {"votes":1,"avg":5,...}	se lee de vuelta
POST con slug falso	→ FOREIGN KEY constraint failed	la lista blanca funciona
POST con dwell 400ms	→ {"error":"voto demasiado rápido"}	el CHECK funciona
GET /admin	→ 401 No autorizado	rechaza sin Access
GET /api/v1/snapshot	→ 401 no autorizado	rechaza sin token

Cómo entrar

1. Abre <https://sably.co/admin>. Cloudflare Access te manda a la pantalla de login del **team domain de la cuenta**, hoy connexis-pages.cloudflareaccess.com.
2. Elige **PIN de un solo uso**, escribe tu correo y pega el código que llega por email. Los correos autorizados están en la política *Administradores de Connexis* de la aplicación **Panel Sably**.
3. Access te devuelve a /admin con la cookie CF_Authorization, y la Function verifica su firma antes de mostrarte nada.

El team domain es de la cuenta, no del proyecto Cloudflare da **un solo** team domain por cuenta y lo comparten todas las aplicaciones de Access. El de Connexis se llamó primero cafeteriaweb-pages y hoy es connexis-pages: por eso la pantalla de login puede mostrar un nombre que no es el del sitio que estás abriendo. No es un despliegue en el proyecto equivocado.

Si se renombra, **hay que actualizar CF_ACCESS_TEAM_DOMAIN y red desplegar**: el valor alimenta a la vez la URL del JWKS y el iss que se espera en el token. Con el valor viejo, el JWKS responde 404 y el panel rechaza a todo el mundo.

Las variables de Pages solo se enlazan en un despliegue nuevo Editarlas en el dashboard **no** afecta al despliegue que ya está sirviendo. Después de tocarlas siempre:

```
gh workflow run deploy-production.yml --ref main
```

Y usa `npx wrangler pages secret put <NOMBRE> --project-name sably` en vez del PATCH de la API: el PATCH **reemplaza** el mapa `env_vars` entero, y ya se llevó por delante todas las variables dos veces.

Pendiente `Turnstile` `Add site` para `sably.co`, y guardar el secreto como `TURNSTILE_SECRET`. Sin él, los envíos de comentarios y leads se rechazan.

Dos hallazgos del despliegue

El adaptador de Astro no sirve hoy. Se probó: `@astrojs/cloudflare 14.2.0` importa `beginContentEntryCollection`, que Astro 7.1.6 ya no exporta, y 14.1.7 no genera salida. El panel va en Pages Functions, que además no tocan el build estático de las 5.918 páginas.

trailingSlash: 'always' interceptaba la API. El handler estático respondía con el `404.html` a `/api/v1/pulso` antes de que la Function lo viera; solo funcionaba con la barra final. Se resuelve con `public/_routes.json` mandando `/api/*` y `/admin*` a las Functions.

Lo que se decidió NO construir ahora

- **Gestión de usuarios:** Access la resuelve, y Filament la cubrirá en Fase 2.
 - **Respuestas anidadas de más de un nivel:** complica la moderación sin aportar.
 - **Notificaciones por comentario nuevo:** el correo de moderación ya avisa.
 - **Búsqueda dentro de comentarios:** con este volumen no hace falta.
-

Operar el día a día

```
# Ver la cola de moderación
wrangler d1 execute sably-pulso --remote \
  --command "SELECT * FROM v_moderacion ORDER BY created_at DESC LIMIT 20"

# Aprobar un comentario a mano
wrangler d1 execute sably-pulso --remote \
  --command "UPDATE comment SET status='approved', moderated_at=unixepoch() WHERE id='<ULID>'"

# Leads de la última semana
wrangler d1 execute sably-pulso --remote \
  --command "SELECT name, email, course_interest, country FROM lead WHERE created_at > unixepoch()-604800"

# Purgar lo que cumplió su plazo
wrangler d1 execute sably-pulso --remote \
  --command "DELETE FROM comment WHERE purge_after < unixepoch(); DELETE FROM lead WHERE purge_after < unixepoch()"
```

También sirve la consola web: **Cloudflare** `Workers & Pages` `D1` `sably-pulso` `Console`, que no requiere instalar nada.

Migrar a Laravel después

Está pensado para que cueste poco:

- Las tablas van **en singular** y con **PK de texto ULID**, que es lo que espera Eloquent con `HasUlid`s.
- Los timestamps son `INTEGER unixepoch UTC` `to_timestamp()` en PostgreSQL.
- El contrato ya es `/api/v1` con los verbos y formas que tendrá Laravel.

Migrar se reduce a exportar el SQL, importarlo en PostgreSQL y cambiar PUBLIC_API_BASE en el frontend. Los componentes del sitio no se tocan.